COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010: Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	9 / 8 / 26	**DATE SUBMITTED:**	9 / 10 / 26
NAME:	Santos, Paterno Miguel R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands On Activity 7.1: Sorting Algorithms (Part 1)

1. PROCEDURE OUTPUT

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

int main() {
    const int maxSize = 100;
    int arrayBubble[maxSize];
    int arraySelection[maxSize];
    int arrayInsertion[maxSize];

    srand(time(0));

    std::cout << "Array for Bubble: \n";
    for (int i = 0; i < maxSize; i++) {
        arrayBubble[i] = rand() % 100;
        std::cout << arrayBubble[i] << " ";
    }

    std::cout << "\n\nArray for Selection: \n";
    for (int i = 0; i < maxSize; i++) {
        arraySelection[i] = rand() % 100;
        std::cout << arraySelection[i] << " ";
    }

    std::cout << "\n\nArray for Insertion: \n";
    for (int i = 0; i < maxSize; i++) {
        arrayInsertion[i] = rand() % 100;
        std::cout << arrayInsertion[i] << " ";
    }

    std::cout << "\n\n";
```

```
Array for Bubble:
27 13 95 30 21 82 89 41 18 72 39 1 14 44 3 1 23 14 66 54 81 72 54 56 91 11 10 14 62 56 45 19 82 81 22 62 48 1 71 87 13 1
4 74 2 69 29 1 44 88 56 29 75 87 19 8 39 34 91 96 54 52 54 15 72 88 86 38 47 82 38 95 8 29 30 43 19 48 23 86 79 23 2 56
50 40 19 67 28 73 74 46 86 47 49 58 1 27 64 63 18

Array for Selection:
80 13 85 48 58 2 63 95 13 5 16 76 53 6 20 59 96 70 69 8 60 40 10 17 88 44 84 26 1 83 20 34 25 62 56 90 70 8 54 0 84 2 25
 73 81 50 60 5 4 83 36 30 20 69 42 44 78 90 94 8 81 75 66 44 66 73 42 62 47 30 35 55 43 28 42 18 87 52 22 49 39 87 14 95
 56 5 0 86 45 93 72 83 87 9 33 20 42 14 29 61

Array for Insertion:
16 11 68 14 68 6 11 68 64 59 52 6 42 56 97 34 21 92 8 56 81 68 58 26 20 11 49 89 57 69 22 63 50 26 47 81 98 80 63 50 14
82 0 21 12 15 34 28 5 6 32 22 7 83 44 86 87 46 86 76 39 70 81 47 64 85 69 87 66 74 20 38 56 39 20 58 9 72 60 92 83 24 39
 75 34 86 73 54 77 78 11 88 17 34 17 34 54 31 70 53
```

Observations

I declared three arrays separately for the three sorting techniques that I am about to use and then it outputs random two digits values which are uniquely different from each array.

Table 7-1. Array of Values for Sort Algorithm Testing

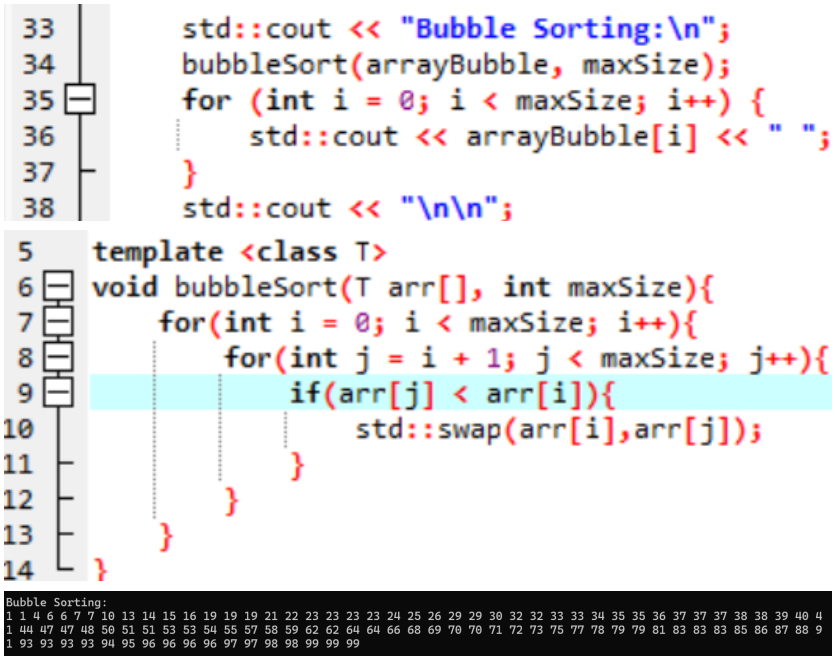
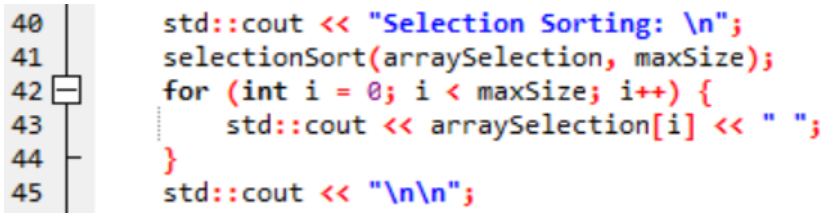
Code + Console Screenshot	
Observations	**It takes the arrayBubble array and the maxSize value to put it through a loop where it enters a nested loop. At the inner loop, it compares two values and when the first value is higher than the second values, then their indexes gets swap using the swap command.**

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	

```cpp
template <class T>
int routine_smallest(T arr[], int k, int arrSize){
    int position;

    T smallestElement = arr[k];

    for(int j = k + 1; j < arrSize; j++ ){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <class T>
void selectionSort(T arr[], int arrSize){
    int min, temp;

    for(int i = 0; i < arrSize - 1; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }
}
```

```
Selection Sorting:
0 0 0 1 2 2 3 4 5 6 8 9 11 12 14 18 18 20 24 24 24 25 26 26 27 29 31 33 34 34 35 36 36 38 62 39 40 41 45 46 46 46 48 50
50 51 52 52 52 53 54 55 56 57 58 59 59 59 59 59 60 62 63 91 67 70 71 72 72 73 75 76 76 77 78 80 80 81 81 81 81 81 84 85
86 97 87 88 88 89 89 90 92 91 92 95 94 95 95 96
```

Observations	**The selection sorting starts by repeatedly finding the smallest element using routine_smallest() and then swapping it with the element at the current position.**

Table 7-3. Selection Sort Algorithm

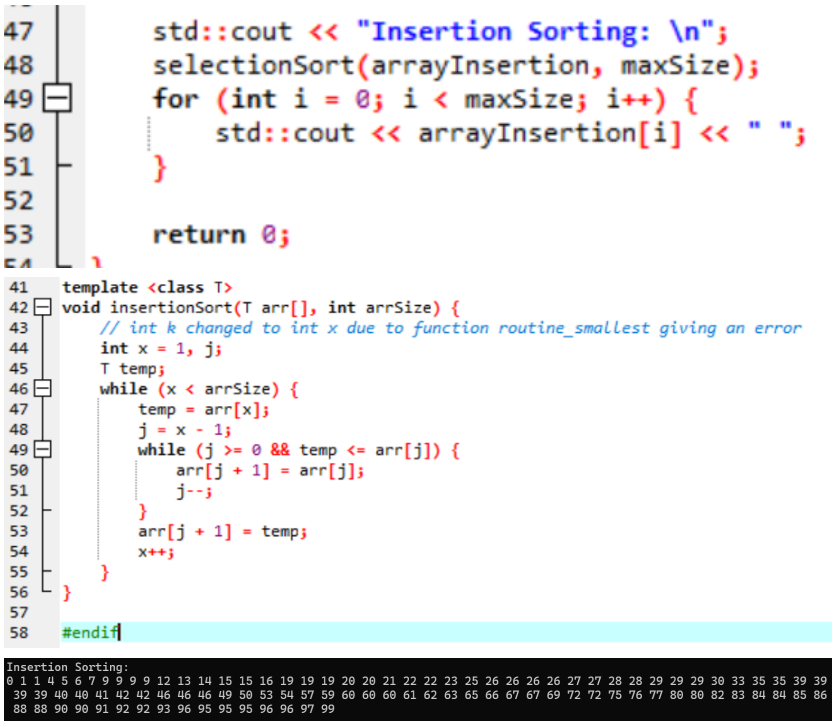
Code + Console Screenshot	
Observations	**Using the second element and comparing it with the elements before it, shifting larger elements one position to the right until the correct position for the current element is found then x acts as the index for the element being inserted while temp temporarily stores the element. The process continues until all elements are placed in their correct positions..**

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Justification:

I chose Insertion Sort because it is simple to use and works well for a small array. Since the array only has values from 1 to 5, the values can be sorted into groups of votes for each candidate.

Algorithm:

1. Randomly generate 100 elements with each value ranging from 1 to 5.
2. Display the unsorted array.
3. Use Insertion Sort to arrange the votes in ascending order.
4. Search through the sorted array.
5. Count how many times each value 1, 2, 3, 4, and 5 occurs.
6. Compare the five vote counts.
7. The candidate with the highest count is the winner.
8. Display the vote count of each candidate and the winning candidate.

Pseudocode:

```
START
    SET maxSize = 100
    DECLARE votes[maxSize]
    GENERATE 100 random votes from 1 to 5

    DISPLAY "Unsorted:"
    FOR i = 0 TO maxSize - 1
        DISPLAY votes[i]
    END FOR

    CALL insertionSort(votes, maxSize)

    DISPLAY "Sorted:"
    FOR i = 0 TO maxSize - 1
        OUTPUT votes[i]
    END FOR

    SET candA = voteCount(votes, maxSize, 1)
    SET candB = voteCount(votes, maxSize, 2)
    SET candC = voteCount(votes, maxSize, 3)
    SET candD = voteCount(votes, maxSize, 4)
    SET candE = voteCount(votes, maxSize, 5)

    DISPLAY "Algorithm Count:"
    DISPLAY candA, candB, candC, candD, candE

    CALL finalRes(candA, candB, candC, candD, candE)
```

```
FUNCTION rngVotes(array, size)
    FOR i = 0 TO size - 1
        SET array[i] to a random number from 1 to 5
    END FOR
END FUNCTION

FUNCTION insertionSort(array, size)
    SET x = 1
    WHILE x < size
        SET temp = array[x]
        SET j = x - 1

        WHILE j >= 0 AND temp < array[j]
            array[j + 1] = array[j]
            j--
        END WHILE

        SET array[j + 1] = temp
        x++
    END WHILE
END FUNCTION


FUNCTION voteCount(array, size, candidate)
    SET count = 0

    FOR i = 0 TO size - 1
        IF array[i] = candidate THEN
            INCREASE count by 1
        END IF
    END FOR
    RETURN count
END FUNCTION

FUNCTION finalRes(a, b, c, d, e)
    SET highest = a
```

```
IF b > highest THEN
    highest = b
END IF

IF c > highest THEN
    highest = c
END IF

IF d > highest THEN
    highest = d
END IF

IF e > highest THEN
    highest = e
END IF

DISPLAY "Winners:"

IF a = highest THEN
    DISPLAY "Bo Dalton Capistrano has", highest, "votes"
END IF

IF b = highest THEN
    DISPLAY "Cornelius Raymon Agustin has", highest, "votes"
END IF

IF c = highest THEN
    DISPLAY "Deja Jayla Banaga has", highest, "votes"
END IF

IF d = highest THEN
    DISPLAY "Lalla Brielle Yabut has", highest, "votes"
END IF

IF e = highest THEN
    DISPLAY "Franklin Relano Castro has", highest, "votes"
END IF
```

```
    END FUNCTION
END
```

Algorithm Code:

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int maxSize = 100;

int main() {

    int votes[maxSize];
    std::srand(std::time(0));
    rngVotes(votes, maxSize);
    std::cout << "Unsorted:\n";
    for (int i = 0; i < maxSize; i++) {
        std::cout << votes[i] << " ";
    }

    insertionSort(votes, maxSize);
    std::cout << "\n\nSorted:\n";
    for (int i = 0; i < maxSize; i++) {
        std::cout << votes[i] << " ";
    }

    int candA = voteCount(votes, maxSize, 1);
    int candB = voteCount(votes, maxSize, 2);
    int candC = voteCount(votes, maxSize, 3);
    int candD = voteCount(votes, maxSize, 4);
    int candE = voteCount(votes, maxSize, 5);

    std::cout << "\n\nAlgorithm Count:\n";
    std::cout << "Candidate 1 (Bo Dalton Capistrano): " << candA << std::endl;
    std::cout << "Candidate 2 (Cornelius Raymon Agustin): " << candB << std::endl;
    std::cout << "Candidate 3 (Deja Jayla Banaga): " << candC << std::endl;
    std::cout << "Candidate 4 (Lalla Brielle Yabut): " << candD << std::endl;
    std::cout << "Candidate 5 Franklin Relano Castro: " << candE << std::endl;

    finalRes<int>(candA, candB, candC, candD, candE);

    return 0;
}
```

```
62  // Supplementary Task:
63  template <class T>
64  void rngVotes(T arr[], int arrSize) {
65      for (int i = 0; i < arrSize; i++) {
66          arr[i] = std::rand() % 5 + 1; // Only allow
67      }
68  }
69
70  template <class T>
71  int voteCount(T arr[], int arrSize, int candidate)
72      int count = 0;
73      for (int i = 0; i < arrSize; i++) {
74          if (arr[i] == candidate) {
75              count++;
76          }
77      }
78      return count;
79  }
80
81  template <class T>
82  void finalRes(int a, int b, int c, int d, int e) {
83      int highest = a;
84      if(b > highest) {
85          highest = b;
86      }
87      if(c > highest) {
88          highest = c;
89      }
90      if(d > highest) {
91          highest = d;
92      }
93      if(e > highest) {
94          highest = e;
```

```
95      }
96
97      std::cout << "\nWinners:\n";
98      if(a == highest) {
99          std::cout << "Bo Dalton Capistrano has " << highest << " votes\n";
100     }
101     if(b == highest) {
102         std::cout << "Cornelius Raymon Agustin has " << highest << " votes\
103     }
104     if(c == highest) {
105         std::cout << "Deja Jayla Banaga has " << highest << " votes\n";
106     }
107     if(d == highest){
108         std::cout << "Lalla Brielle Yabut has " << highest << " votes\n";
109     }
110     if(e == highest){
111         std::cout << "Franklin Relano Castro has " << highest << " votes\n"
112     }
113 }
114 #endif
```

Output Testing:

```
Unsorted:
1 3 2 2 1 2 5 5 1 2 5 3 2 5 2 2 2 2 2 1 3 3 1 3 2 1 3 5 2 3 4 2 3 5 3 1 5 1 1 2 3 5 3 5 4 1 2 4 1 1 5 2 5 4 5 1 2 1 4 5
5 2 5 3 2 1 1 2 3 1 3 3 1 3 3 1 3 1 5 5 5 1 2 3 4 3 5 5 5 2 3 3 2 1 4 3 1 3 4 1

Sorted:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

Algorithm Count:
Candidate 1 (Bo Dalton Capistrano): 24
Candidate 2 (Cornelius Raymon Agustin): 23
Candidate 3 (Deja Jayla Banaga): 24
Candidate 4 (Lalla Brielle Yabut): 8
Candidate 5 Franklin Relano Castro: 21

Winners:
Bo Dalton Capistrano has 24 votes
Deja Jayla Banaga has 24 votes

--------------------------------
Process exited after 2.049 seconds with return value 0
Press any key to continue . . .
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
		Algorithm Count: Candidate 1 (Bo Dalton Capistrano): 12 Candidate 2 (Cornelius Raymon Agustin): 18 Candidate 3 (Deja Jayla Banaga): 27 Candidate 4 (Lalla Brielle Yabut): 20 Candidate 5 Franklin Relano Castro: 23

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.